

Introdução aos Padrões de Projeto

Curso de Desenvolvimento de Software Multiplataforma (DSM)

Disciplina: Técnicas de Programação II (TPII)

Professora: Waldinelly Costa

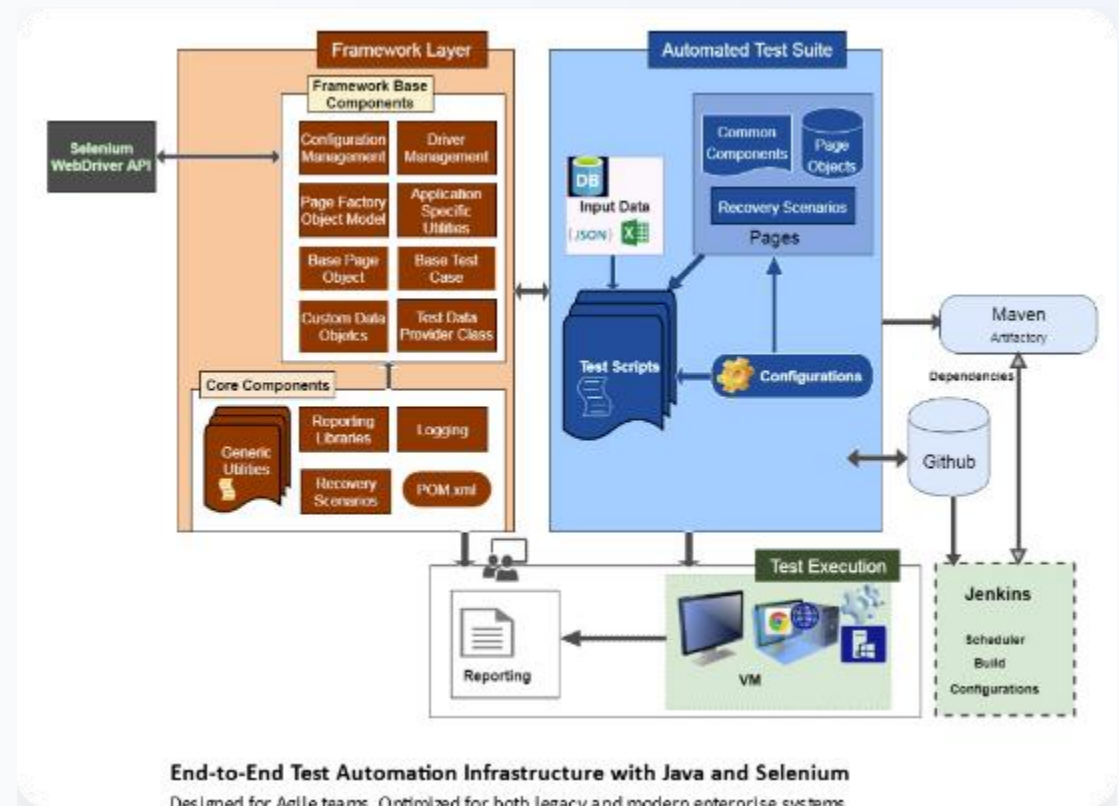
Fatec Indaiatuba • Agosto de 2026

Fundamentos

O que são Padrões de Projeto (*Design Patterns*) e por que eles são absolutamente essenciais na arquitetura de software moderno?

Definição Formal

- ▶ Padrões de projeto são **soluções típicas** e comprovadas para problemas comuns em design de software.
- ▶ Funcionam como **plantas arquitetônicas** pré-fabricadas, que você pode personalizar para resolver problemas de design recorrentes.
- ▶ Não são trechos de código copiáveis, mas sim **conceitos gerais** para resolver um problema específico.
- ▶ O detalhamento da implementação depende da realidade do seu projeto e da linguagem de programação.



Por que utilizá-los?

100%

Linguagem Universal

Comunicação e Reuso

Os padrões fornecem um vocabulário padrão para a equipe de desenvolvimento.

Ao invés de explicar toda a lógica de uma arquitetura, você diz: *"Vamos usar um Singleton aqui"*, e todos compreendem imediatamente o conceito, acelerando a comunicação e mitigando falhas estruturais.

As 3 Categorias de Padrões



Criacionais

Lidam com os mecanismos e lógicas de criação de objetos, aumentando a flexibilidade e reutilização do código.



Estruturais

Lidam com a composição de classes e objetos para formar estruturas maiores, mantendo a eficiência.



Comportamentais

Focam nos algoritmos e na atribuição rigorosa de responsabilidades e comunicação entre os objetos.

Padrões Criacionais

Abstraem o processo de instanciação. Eles ajudam a tornar um sistema independente de como seus objetos são criados, compostos e representados.

O foco é ocultar a lógica de criação complexa do usuário final.

Ex: Singleton, Factory Method, Builder



**D. Software Development,
ep at a Time**

k Bailey

Padrões Estruturais



Preocupam-se com a forma como classes e objetos são **compostos para formar estruturas maiores e complexas**.

Eles garantem que, se uma parte específica do sistema mudar, a estrutura inteira não precise ser alterada ou reescrita, promovendo alta flexibilidade.

Exemplos Clássicos:

Adapter, Decorator, Facade, Composite.

Padrões Comportamentais

Focam estritamente nos **algoritmos e na atribuição de responsabilidades** entre os objetos.

Eles descrevem não apenas os padrões de objetos ou classes, mas também detalham os **padrões de comunicação** entre eles de maneira eficiente e desacoplada.

Exemplos: Observer, Strategy, Command, Iterator.



“

Cada padrão descreve um problema no nosso ambiente e o cerne da sua solução, de forma que você possa usar essa solução mais de um milhão de vezes sem repetição.

— Christopher Alexander (Inspiração original do GoF)

Referências Bibliográficas



GAMMA, Erich et al. (Gang of Four - GoF)

Padrões de Projeto: Soluções Reutilizáveis de Software Orientado a Objetos. Bookman, 2000. (*A obra fundamental*).



FREEMAN, Eric; ROBSON, Elisabeth

Use a Cabeça! Padrões de Projetos. Alta Books. (*Uma abordagem didática e acessível*).



REFACTORING.GURU

Catálogo online interativo e detalhado de Design Patterns. Disponível em: refactoring.guru/pt-br/design-patterns

Padrões Arquiteturais: MVC, MVP e MVVM

Separando responsabilidades e criando softwares escaláveis e manuteníveis.

Prof.^a Waldinelly Costa

Curso de Desenvolvimento de Software Multiplataforma (DSM)

Disciplina: Técnicas de Programação II (TPII)

Fatec Indaiatuba • Agosto de 2026

A Base da Arquitetura Visual

Por que precisamos organizar o código em camadas lógicas? O princípio da **Separação de Preocupações** (Separation of Concerns).

Os 3 Pilares das Interfaces



O Modelo (Model)

Representa o coração do aplicativo. Contém os **dados** reais e todas as **regras de negócio** fundamentais. Não sabe nada sobre como a interface é desenhada.



A Visão (View)

É a camada visual, a **Interface do Usuário (UI)**. É o que o usuário enxerga e onde ele clica. Sua única função deveria ser exibir dados e capturar ações.



O Mediador

O meio de campo (Controller, Presenter ou ViewModel). Ele recebe as ações da View, pede os dados ao Model e devolve a resposta formatada para a View.

MVC (Model-View-Controller)

- ▶ O pioneiro! Amplamente utilizado no desenvolvimento Web clássico e frameworks backend modernos.
- ▶ O **Controller** recebe a requisição do usuário (como um acesso a uma URL).
- ▶ O Controller solicita e atualiza os dados no **Model**.
- ▶ O Controller então seleciona qual **View** (tela HTML) será renderizada e enviada de volta ao usuário com esses dados.

Exemplo MVC: A Analogia do Restaurante

Como Funciona?

Imagine que você vai a um restaurante.

- ▶ **O Cliente:** É o Usuário do sistema.
- ▶ **O Garçom (Controller):** Ele anota seu pedido, leva até a cozinha e decide em qual mesa servir.
- ▶ **O Cozinheiro (Model):** Fica na cozinha, tem a receita (regras de negócio) e os ingredientes (dados). Faz o prato.
- ▶ **O Prato Final (View):** É a apresentação bonita da comida que chega na sua mesa.

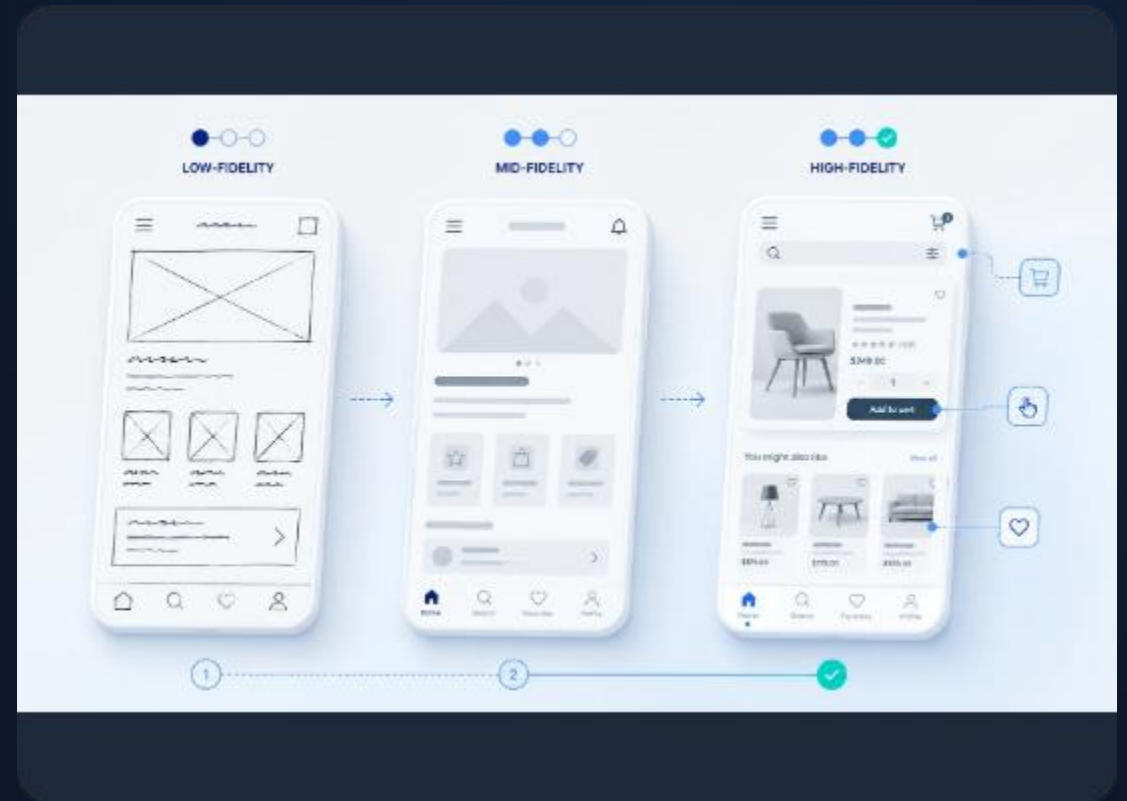
Por que usar?

Se o dono do restaurante quiser mudar as louças (a View) para pratos quadrados, o cozinheiro (Model) não precisa reaprender a cozinhar.

E se o cozinheiro mudar de marca de arroz (Model), o cliente continuará usando os mesmos pratos e garfos (View). A independência facilita a evolução do sistema!

MVP (Model-View-Presenter)

- ▶ Muito popular na construção de aplicativos **Mobile (Android nativo)** clássicos e sistemas Desktop.
- ▶ A principal diferença: A View se torna extremamente "**burra**" e **passiva**. Ela não tem lógica alguma.
- ▶ A View nunca se comunica com o Model.
- ▶ O **Presenter** assume 100% da inteligência e manipula a View diretamente através de interfaces.



Exemplo MVP: O Formulário de Login

A View (Tela do Celular)

A View possui dois campos (usuário, senha) e um botão. Quando o usuário clica, ela **não** verifica se a senha é válida. Ela apenas grita para o Presenter:

"Ei Presenter, o usuário clicou em 'Entrar', o texto é X e a senha é Y. O que eu faço agora?"

O Presenter (O Cérebro)

O Presenter recebe essa informação, repassa ao **Model** (que consulta o banco de dados) e recebe a validação.

O Presenter então dá uma **ordem direta** para a View:

"View, oculte o botão, mostre uma barra de carregamento e pinte o texto de vermelho avisando erro."

MVVM (Model-View-ViewModel)



É o padrão dominante nas tecnologias modernas (Angular, React, Vue, Swift, Android Moderno).

Funciona sob o conceito de **Data Binding (Vinculação de Dados) e Reatividade**.

A View se "amarra" às variáveis do **ViewModel**. Quando uma variável muda de valor no ViewModel, a tela pisca e atualiza sozinha, sem ninguém mandar!

⚡ Exemplo Prático:

Num carrinho de compras, se o ViewModel alterar a variável "total" de R\$10 para R\$50, o texto na tela atualiza automaticamente (Magia da Reatividade).

Referências Bibliográficas



FOWLER, Martin.

Patterns of Enterprise Application Architecture. Addison-Wesley, 2002. (A Bíblia da arquitetura corporativa e MVC clássico).



MICROSOFT Docs.

The MVVM Pattern. Padrões oficiais de documentação para arquiteturas reativas e vinculação de dados.



MARTIN, Robert C. (Uncle Bob).

Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.